

Chap.3 옵티마이저 (Optimizer)

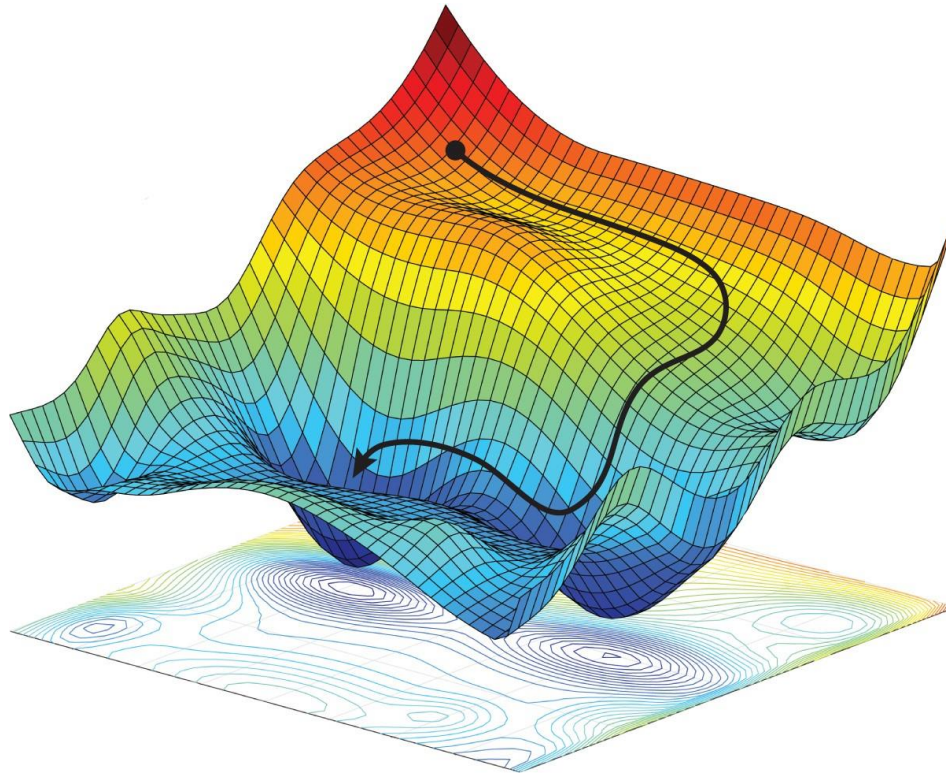
방 수 식 교수
(bang@tukorea.ac.kr)

한국공학대학교 전자공학부

2024년도 2학기
고급머신러닝실습 & 인공지능설계실습2

- 옵티마이저

- 비용함수를 최소화 시키는 최적의 모델 파라미터를 찾아주는 알고리즘

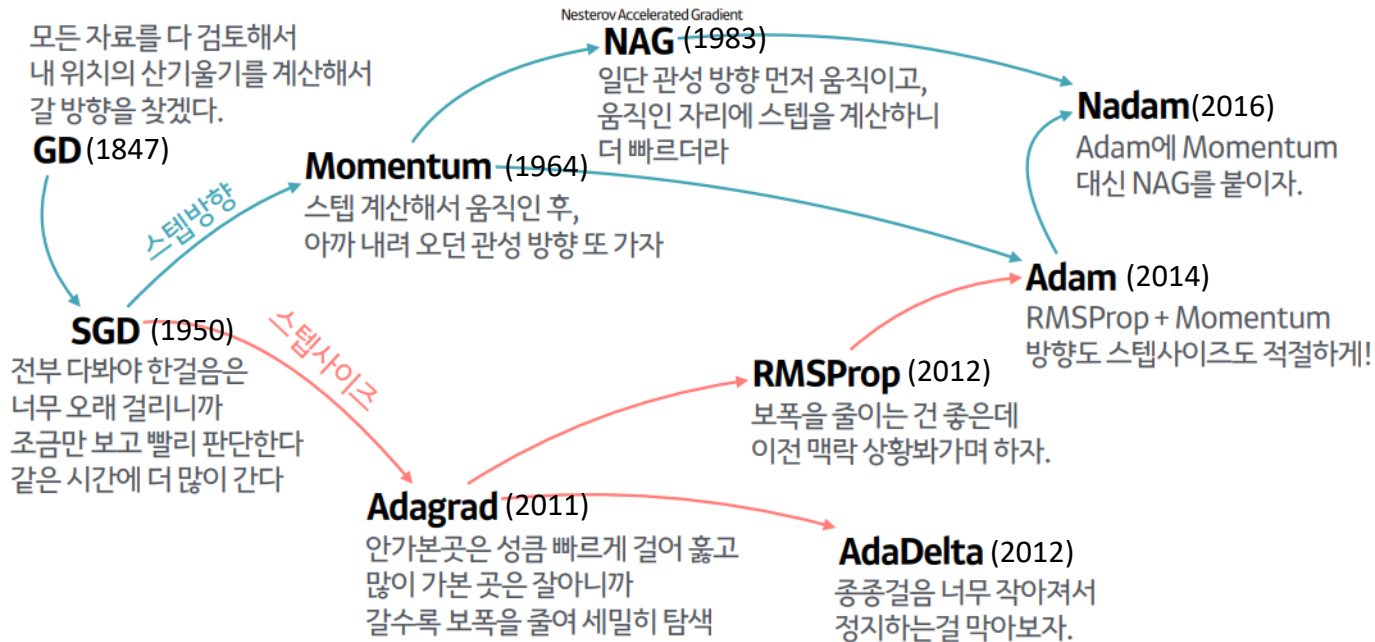


출처 : https://codetorial.net/tensorflow/basics_of_optimizer.html

■ 다양한 옵티마이저

- 최적의 모델 파라미터로 가는 경로는 다양하기 때문에
옵티마이저도 여러 알고리즘으로 개발

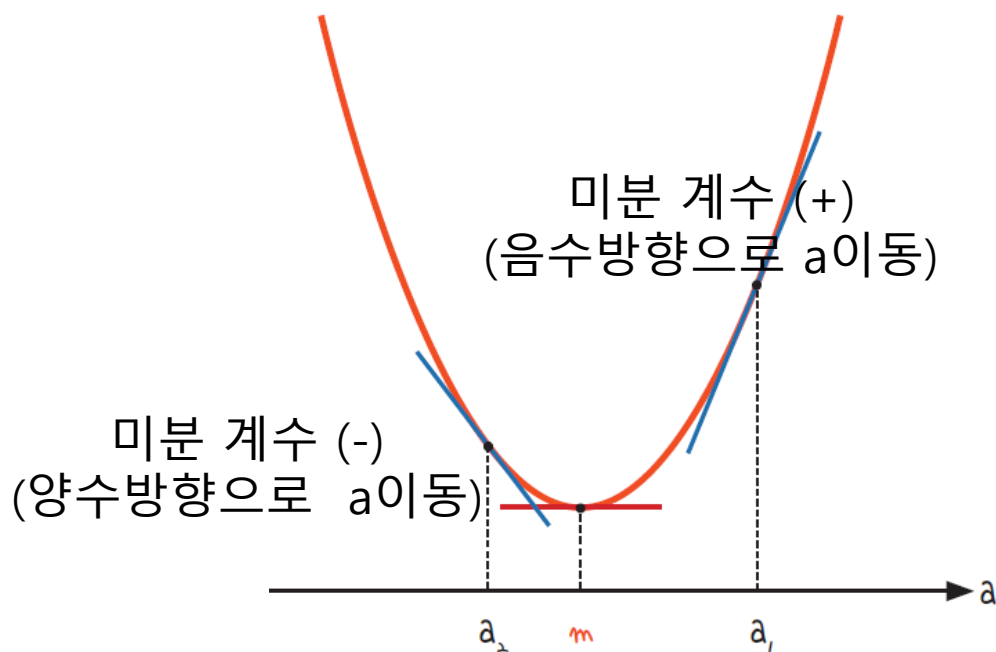
산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



경사하강법(Gradient Decent Method)



- 최소값에서 접선의 기울기에 해당하는 미분 계수는 0
 - 여러 번의 반복 (iteration)을 통해 미분 계수가 0이 되는 지점을 찾는다.
- 미분 계수(Gradient)는 언제나 현재 위치에서 함수값이 커지는 방향을 가리키므로, **미분 계수의 반대 방향으로 이동**하면 최소값을 찾을 수 있음
 - 즉 Gradient에 마이너스(-)를 취하여 더한다.



Learning Rate

$$\boxed{\text{New}} w_0[t+1] = \boxed{\text{Old}} w_0[t] - \boxed{\alpha} \frac{\partial}{\partial w_0} \epsilon_{MSE}(\boxed{\text{Old}} w_0, w_1)$$
$$w_1[t+1] = w_1[t] - \alpha \frac{\partial}{\partial w_1} \epsilon_{MSE}(w_1, w_2)$$

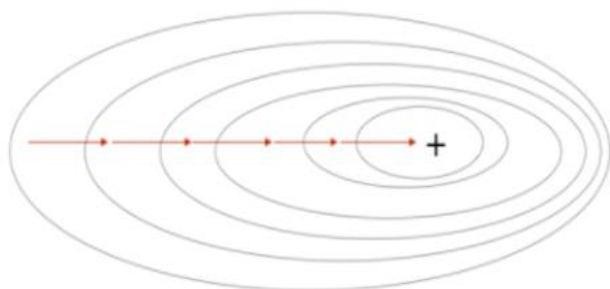
확률적 경사하강법(SGD)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

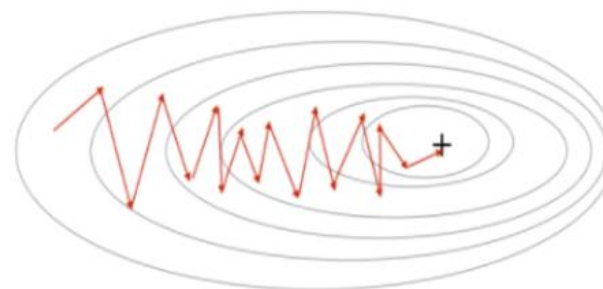
- Batch: 경사하강법에서 기울기를 계산하는데 사용하는 데이터의 갯수
- 확률적 경사하강법(Stochastic Gradient Descent, SGD)

Gradient Decent



Batch Size: N (데이터 전체)

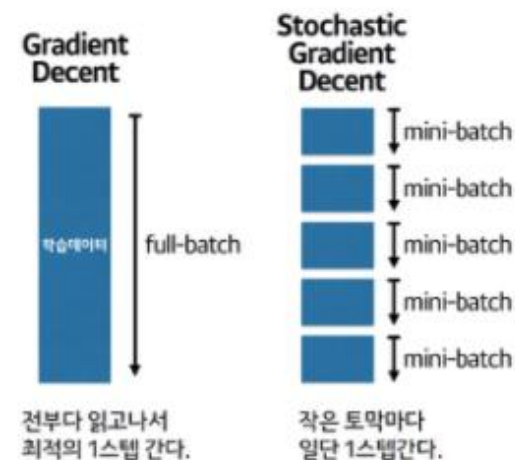
Stochastic Gradient Decent



Batch Size: 1

출처: <https://engmrk.com/mini-batch-gd/>

- Mini Batch: 일부 모음으로 SGD를 적용하는 방법



■ 경사하강법(Gradient Decent Method)

- 데이터 셋이 큰 경우 연산량이 매우 많음
 - 수렴하는데 굉장히 많은 시간이 소요됨

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$

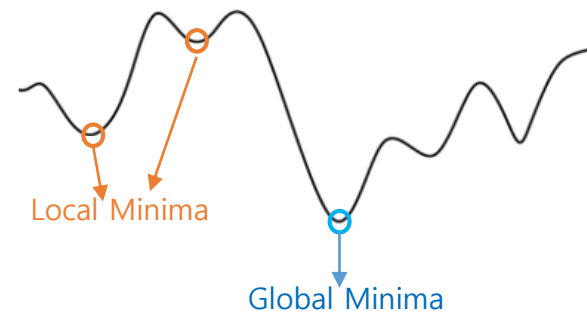
θ : parameter set of model η : Learning rate $J(\theta)$: Loss function

■ 확률적 경사하강법(Stochastic Gradient Descent, SGD)

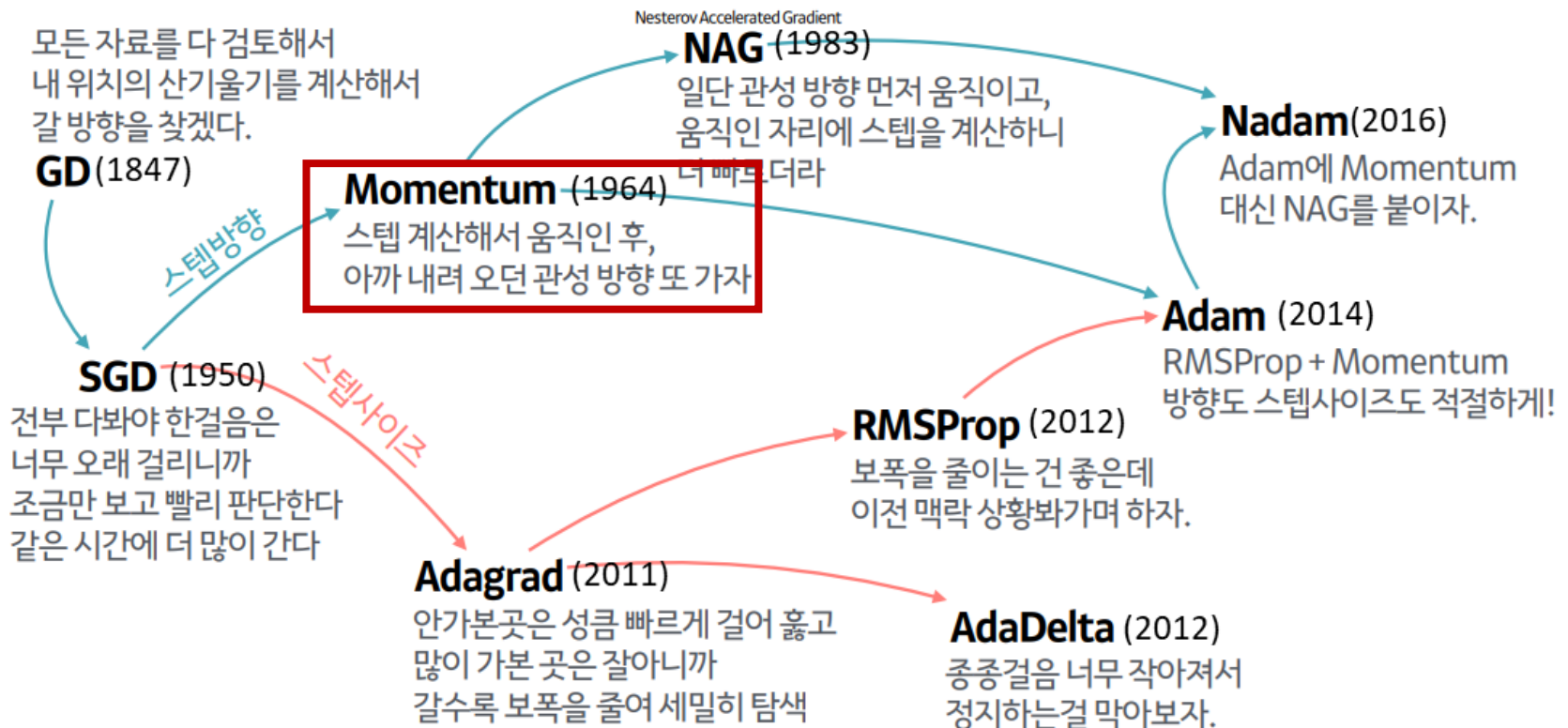
- 모든 데이터가 아닌 일부 샘플 데이터만 사용하여
경사하강법에 비해 연산시간이 빠름
- Local Minima에 취약함

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t; \mathbf{x}^{(i)}; \mathbf{y}^{(i)})$$

$\mathbf{x}$: training data $\mathbf{y}$: label i : index



산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



■ Momentum

- Momentum은 **최적의 방향으로 SGD가 가속하도록** 도와주는 알고리즘
- γm_{t-1} 항이 내리막에선 가속하고, 방향이 바뀔 때 파라미터 **갱신량 감소**
- Local Minima는 잘 빠져나오지만 수렴이 어려움



(a) SGD without momentum



(b) SGD with momentum

출처 : Sebastian Ruder, "An overview of gradient decent optimization algorithms"

GD

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$



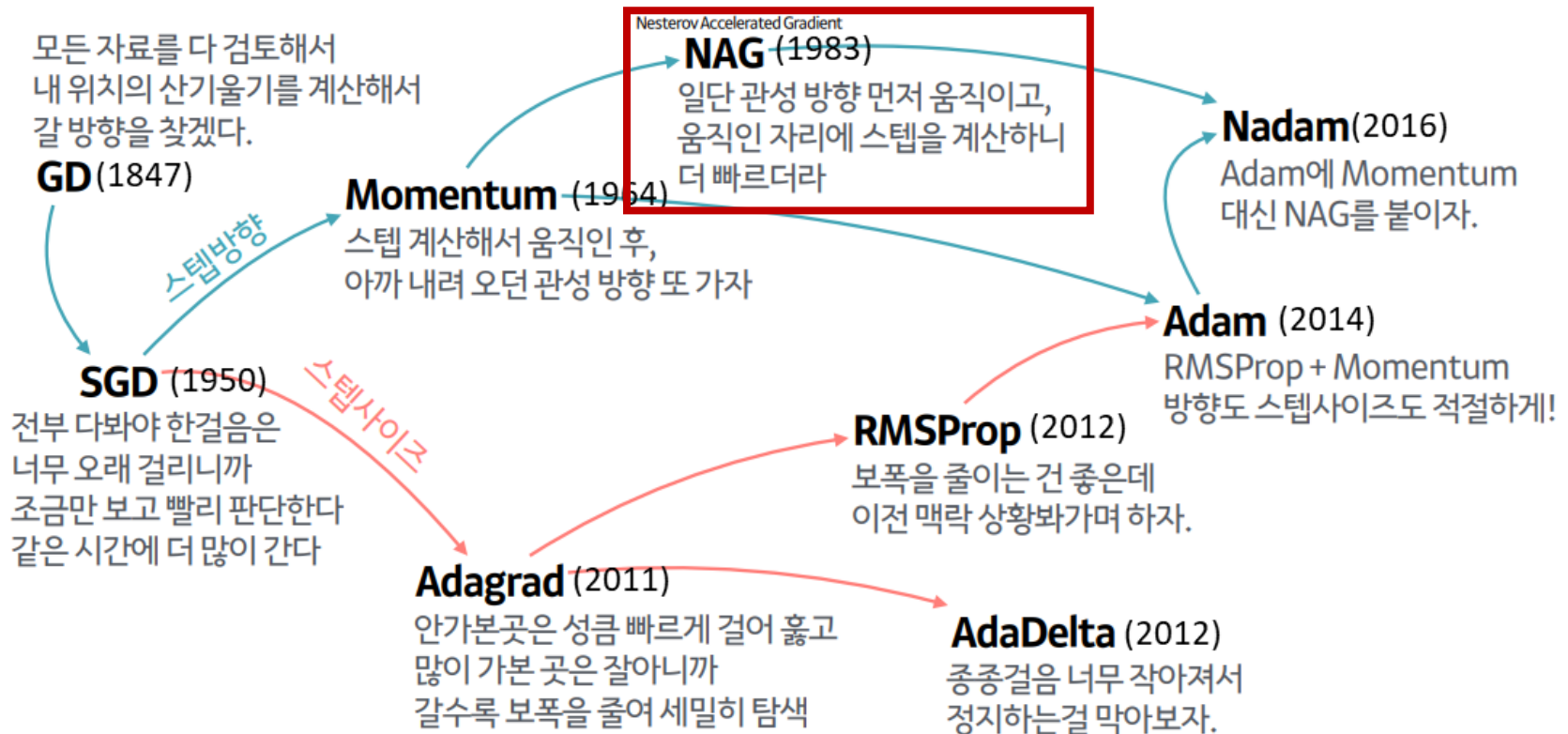
Momentum

$$m_t = \gamma m_{t-1} + \eta \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - m_t$$

γ : 모멘텀 상수

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



NAG(Nesterov Accelerated Gradient Descent)

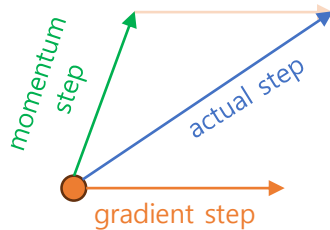


지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ NAG(Nesterov Accelerated Gradient Descent)

- NAG는 멈춰야 할 시기에 더 나아가지 않고 제동을 거는 방법
- Momentum 방식을 베이스로 현재 미분계수(gradient)가 아닌, momentum step 이후의 gradient를 통해 업데이트
- Momentum에 비해 빠르게 수렴하는 경향이 있음.

Momentum update



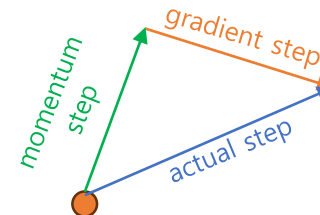
Momentum

$$m_t = \gamma m_{t-1} + \eta \nabla_{\theta} J(\theta)$$

$$\theta_{t+1} = \theta_t - m_t$$



NAG update



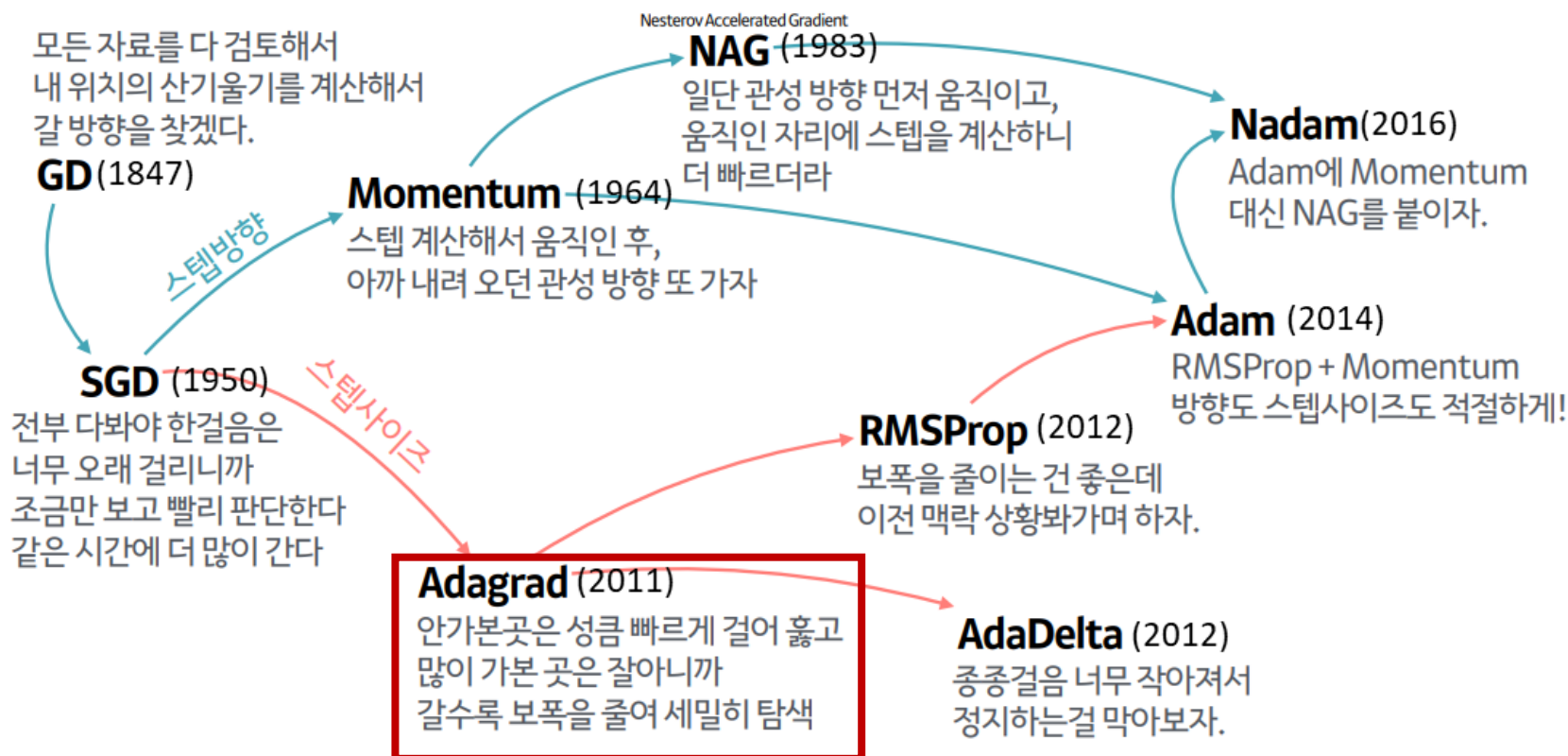
NAG

$$m_t = \gamma m_{t-1} + \eta \nabla_{\theta} J(\theta_t - \gamma m_{t-1})$$

$$\theta_{t+1} = \theta_t - m_t$$

Momentum 방향으로
더 나아간 위치

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보

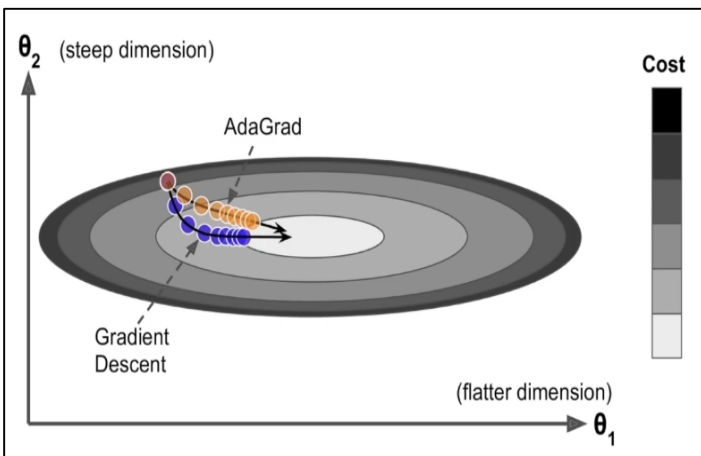


Adagrad (Adaptive Gradients)

Adagrad (Adaptive Gradients)

- 모든 매개변수가 동일하게 갱신되는 것이 아니라, 매개변수의 **빈도수에 따라 가중치를 부여**하여 갱신
- Adagrad는 매번 각각의 매개변수에 다른 학습률(learning rate, lr)을 사용
- 초기 lr (η)를 설정하면 **학습과정에서 G_t 가 자동으로 lr를 조정**
- 학습이 진행될수록 G_t 의 값이 무한히 커져 lr이 0에 수렴해 업데이트가 안됨

$\odot$ 는 element-wise 연산



GD

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t)$$



Adagrad

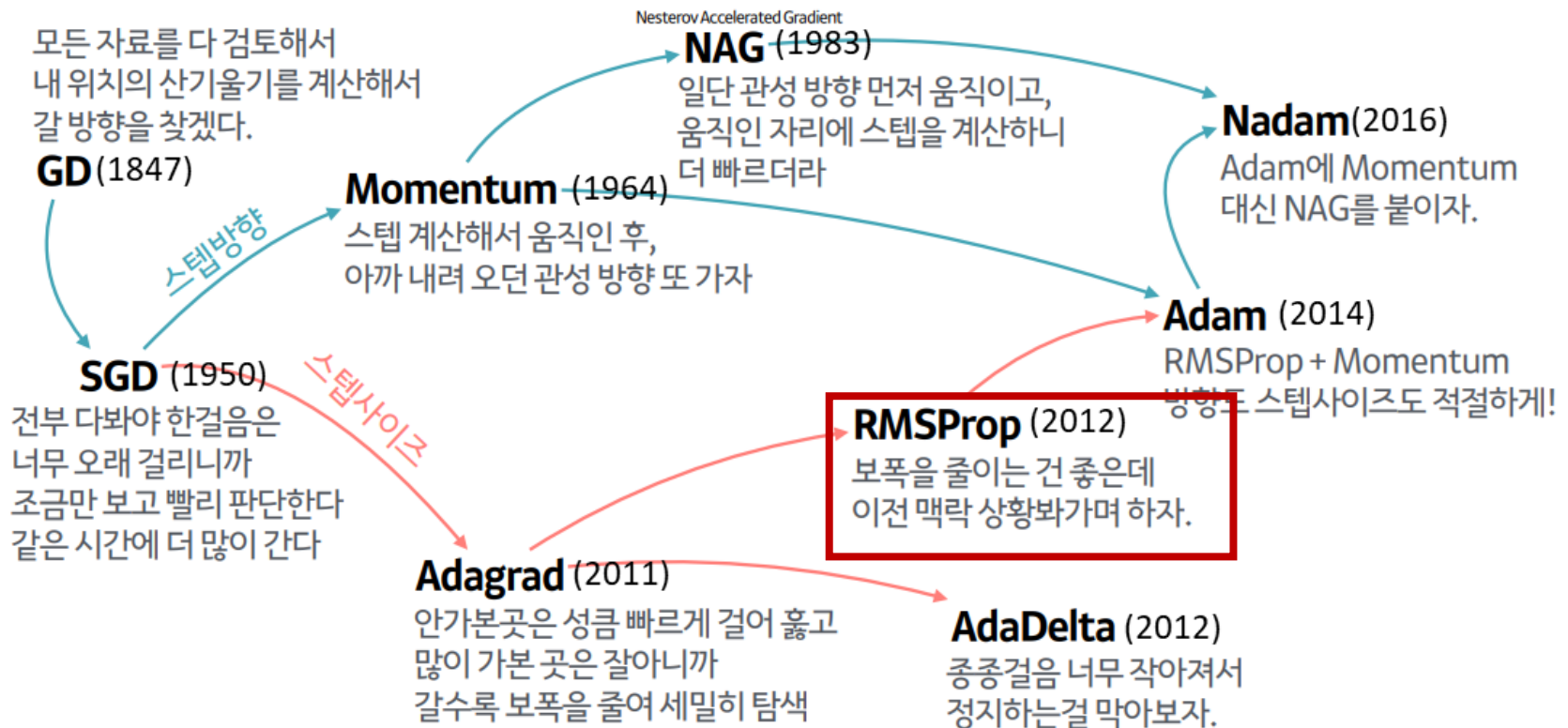
$$\begin{aligned} g_{t,i} &= \nabla_{\theta} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \varepsilon}} \odot g_{t,i} \\ G_{t,i} &= G_{t-1,i} + g_{t,i} \odot g_{t,i} \end{aligned}$$

g_t : Adaptive learning rate ($k \times 1$ 행렬)

ε : 매우 작은 값

왜 필요할까?

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



RMSProp (Root Mean Square Propagation)



■ RMSProp (Root Mean Square Propagation)

- Adagrad에서 문제점을 개선하기 위해 G_t 계산식에 **지수 이동 평균을 적용함**
- 지수 이동 평균이란?
 - 최근 데이터에 가중치를 더 높게 주어 과거와 현재 데이터의 가중치를 **지수적으로 감소시키며 평균을 계산하는 방법**
- Gradient 제곱의 지수 이동 평균을 기대 값이라 표현함

Adagrad

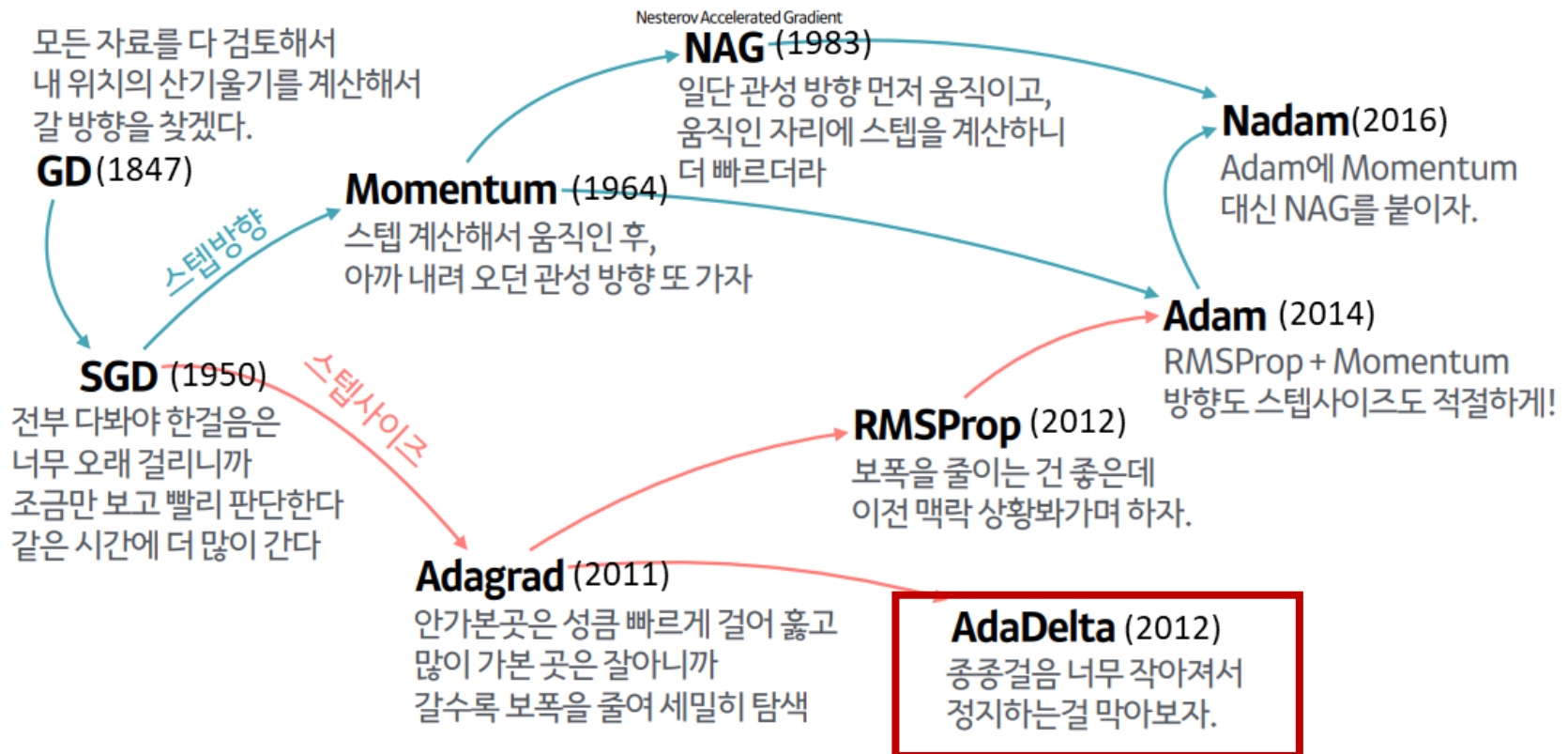
$$\begin{aligned} g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,i} + \varepsilon}} \odot g_{t,i} \\ G_{t,i} &= G_{t-1,i} + g_{t,i} \odot g_{t,i} \end{aligned}$$



RMSProp

$$\begin{aligned} g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_t + \varepsilon}} \odot g_{t,i} \\ G_t &= \gamma G_t + (1 - \gamma) g_{t,i} \odot g_{t,i} \end{aligned}$$

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



AdaDelta (Adaptive Delta)



■ AdaDelta (Adaptive Delta)

- RMSProp와 동일하게 g_t 계산식에 지수 이동 평균을 적용함
- Step size를 단순히 η 로 사용하는 것이 아닌
step size 제곱의 지수 이동 평균을 사용함
 - unit을 맞춰주기 위함

RMSProp

$$\begin{aligned}g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_t + \varepsilon}} \odot g_{t,i} \\ G_t &= \gamma G_t + (1 - \gamma) g_{t,i} \odot g_{t,i}\end{aligned}$$



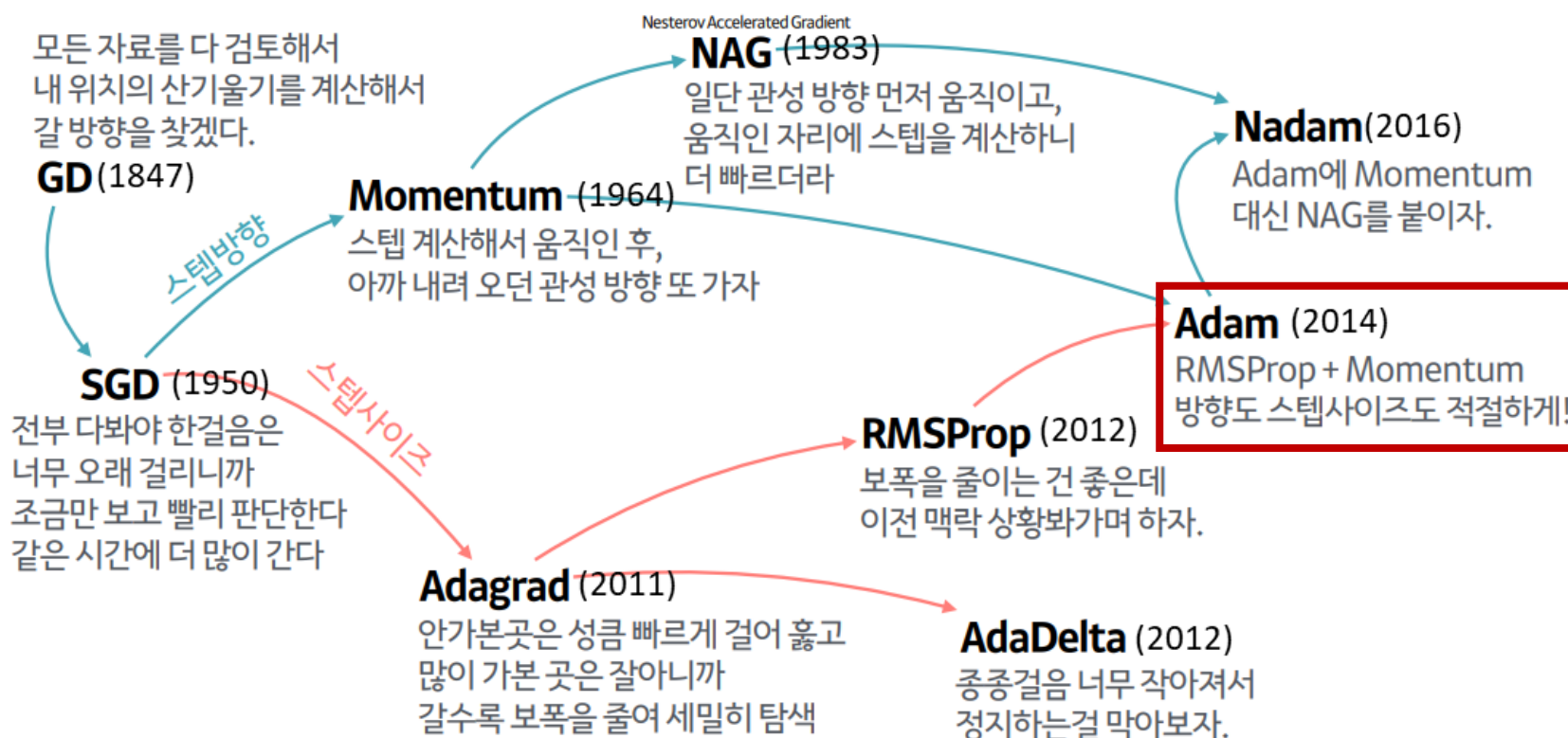
AdaDelta

$$\begin{aligned}E[g^2]_t &= \gamma E[g^2]_{t-1} + (1 - \gamma) g_t^2 \\ E[\Delta\theta^2]_t &= \gamma E[\Delta\theta^2]_{t-1} + (1 - \gamma) \Delta\theta_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\sqrt{E[\Delta\theta]_{t+\varepsilon}}}{\sqrt{E[g^2]_t + \varepsilon}} \odot g_{t,i}\end{aligned}$$

$$\overset{u}{\boxed{\theta_{t+1}}} = \overset{u}{\boxed{\theta_t}} - \eta \overset{u^{-1}}{\boxed{\nabla_{\theta} J(\theta_t)}}$$

$$\overset{u}{\boxed{\Delta\theta_t}} = \frac{\overset{u}{\sqrt{E[\Delta\theta]_t + \varepsilon}} * \overset{u^{-1}}{\boxed{g_{t,i}}}}{\overset{u^{-1}}{\sqrt{E[g^2]_t + \varepsilon}}}$$

산 내려오는 작은 오솔길 찾기(Optimizer)의 발달 계보



Adam (Adaptive Momentum Estimation)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

Adam (Adaptive Momentum Estimation)

- RMSProp+ Momentum
- $\hat{m}_t$ 는 momentum 역할, $\hat{v}_t$ 는 파라미터에 서로 다른 lr를 적용하는 역할
- 대부분의 상황에서 준수한 성능을 보이기 때문에 가장 많이 사용되는 옵티마이저
- Adam의 저자는 $\beta_1 = 0.9$ $\beta_2 = 0.999$ 로 제안

Momentum

$$\begin{aligned} m_t &= \gamma m_{t-1} + \eta \nabla_{\theta} J(\theta) \\ \theta_{t+1} &= \theta_t - m_t \end{aligned}$$

RMSProp

$$\begin{aligned} g_{t,i} &= \nabla_{\theta_t} J(\theta_{t,i}) \\ \theta_{t+1,i} &= \theta_{t,i} - \frac{\eta}{\sqrt{G_t + \varepsilon}} \odot g_{t,i} \\ G_t &= \gamma G_t + (1 - \gamma) g_{t,i} \odot g_{t,i} \end{aligned}$$

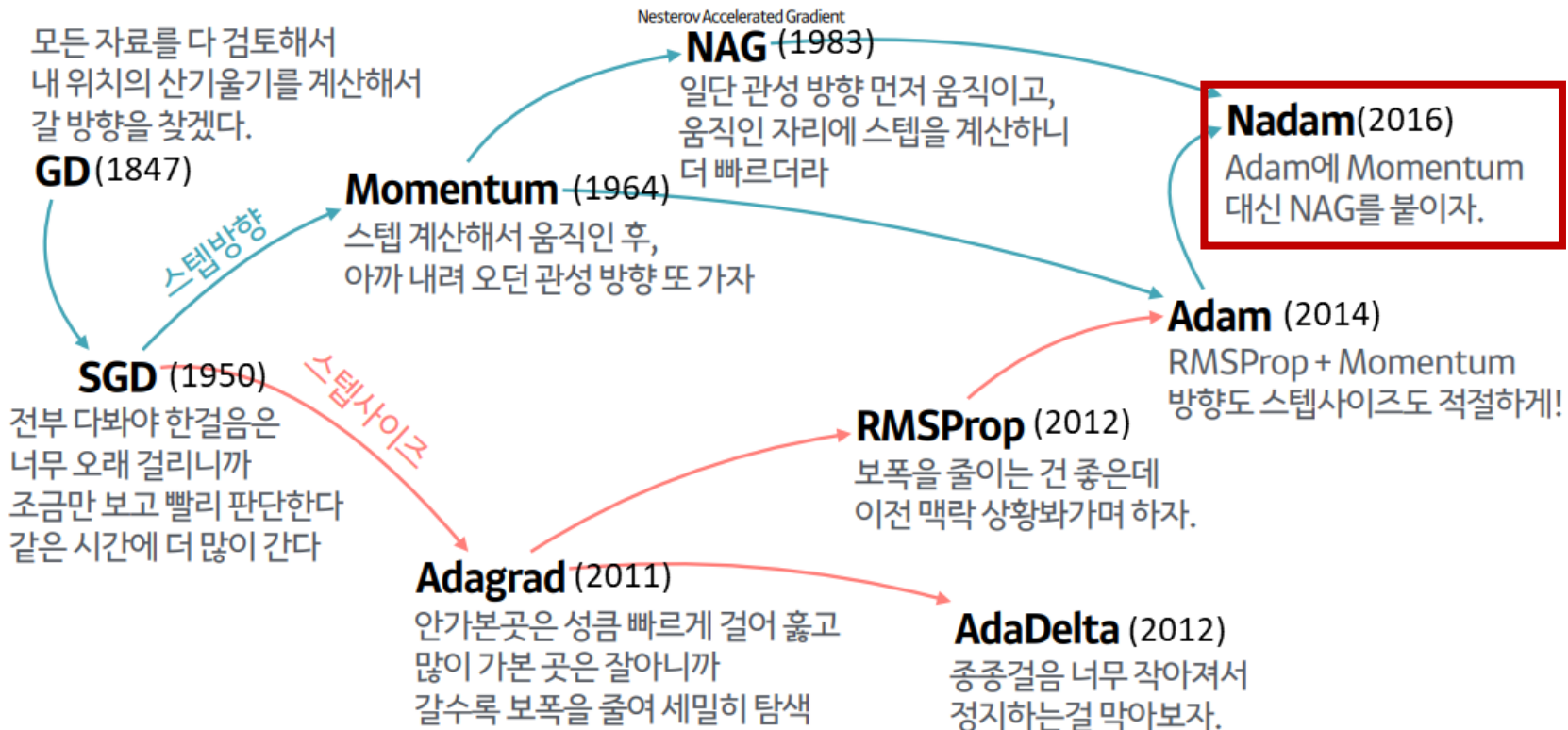


Adam

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \odot \hat{m}_t \end{aligned}$$

첫 step에서 m_t 와 g_t 의 반영비율을 맞춰주기 위해 (트릭사용)

산 내려오는 작은 오솔길 잘찾기(Optimizer)의 발달 계보



Nadam (Nesterov Accelerated Adaptive Moment Estimation)



지능형 예측·진단 연구실
Intelligent Prognostics & Diagnostics Lab.

■ Nadam (Nesterov Accelerated Adaptive Moment Estimation)

- RMSProp+ NAG

Momentum

$$\begin{aligned} m_t &= \gamma m_{t-1} + \eta \nabla_{\theta} J(\theta) \\ \theta_{t+1} &= \theta_t - m_t \\ \theta_{t+1} &= \theta_t - (\gamma m_{t-1} + \eta g_t) \end{aligned}$$



Modify NAG

$$\begin{aligned} g_t &= \eta \nabla_{\theta} J(\theta_t) \\ m_t &= \gamma m_{t-1} - \eta g_t \\ \theta_{t+1} &= \theta_t - (\gamma m_t + \eta g_t) \end{aligned}$$

Adam

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\ \hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \end{aligned}$$



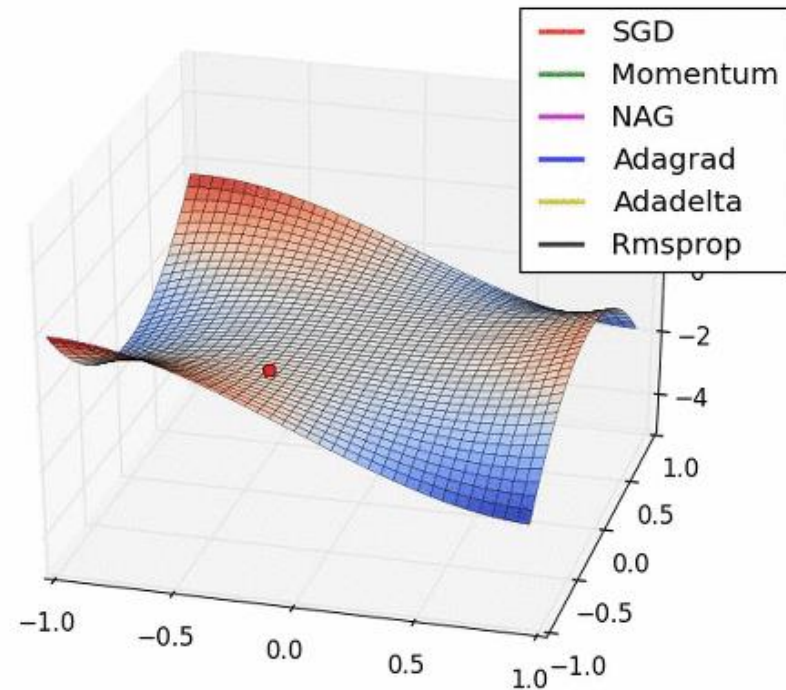
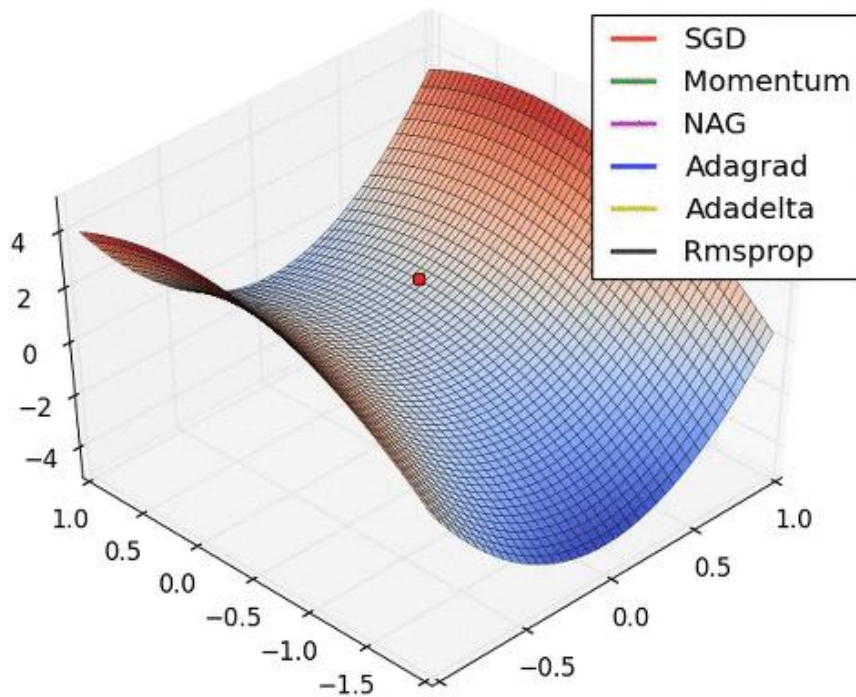
Nadam

$$\begin{aligned} \theta_{t+1} &= \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \frac{\beta_1 m_{t-1} + (1 - \beta_1) g_t}{1 - \beta_1^t} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \left(\beta_1 \hat{m}_{t-1} + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t} \right) \text{여기까지 Adam수식} \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \left(\beta_1 \hat{m}_t + \frac{(1 - \beta_1) g_t}{1 - \beta_1^t} \right) \text{Adam} \Rightarrow \text{Nadam} \end{aligned}$$

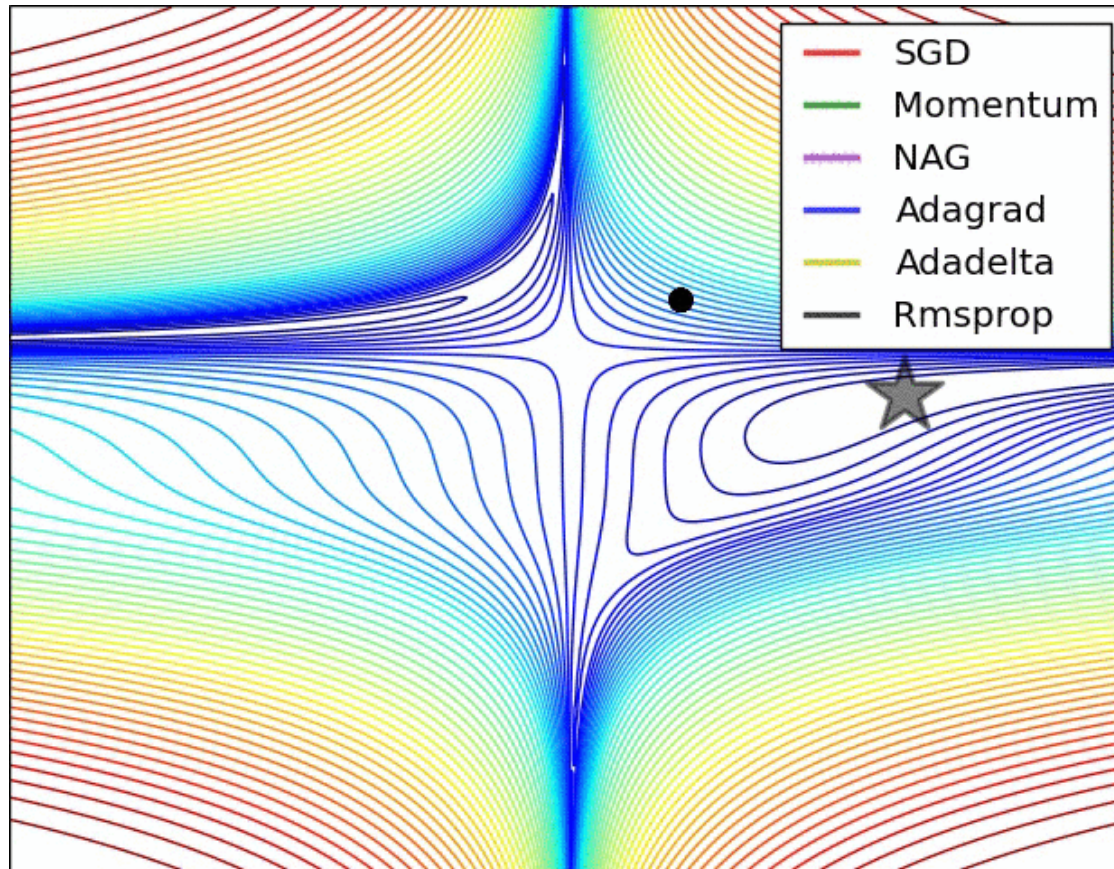
Optimizer



Optimizer 비교 1



Optimizer 비교 2



loss 평면 등고선에서 SGD optimization

출처 : Sebastian Ruder, "An overview of gradient decent optimization algorithms"

■ Keras Optimizer 설정

```
optimizer = keras.optimizers.SGD(lr = 0.1)  
model.compile(loss='categorical_crossentropy',  
              optimizer = optimizer, metrics=["accuracy"])
```

Optimaizer 설정

learning rate 설정

손실함수 설정

Optimizers
SGD
RMSprop
Adam
AdamW
Adadelata
Adagrad

momentum은?



```
tf.keras.optimizers.SGD(  
    learning_rate=0.01,  
    momentum=0.0,  
    nesterov=False,  
    weight_decay=None,  
    clipnorm=None,  
    clipvalue=None,  
    global_clipnorm=None,  
    use_ema=False,  
    ema_momentum=0.99,  
    ema_overwrite_frequency=None,  
    jit_compile=True,  
    name="SGD",  
    **kwargs  
)
```

momentum 설정 예시

```
optimizer = keras.optimizers.SGD(lr=0.001,  
                                  momentum=0.9)
```

NAG 설정 예시

```
optimizer = keras.optimizers.SGD(lr=0.001,  
                                  momentum=0.9,  
                                  nesterov=True)
```

사용 가능한 Keras Optimizer

Keras 홈페이지 SGD 설명

Optimizer 이외에도 여러 설정법 참고 가능

Keras Optimizer 설명 주소 : <https://keras.io/api/optimizers/>

Chap.2 실습



각 실습문제에 대해 code(+ 주석), 그래프, 분석 내용 등은 필수적으로 포함시킬 것

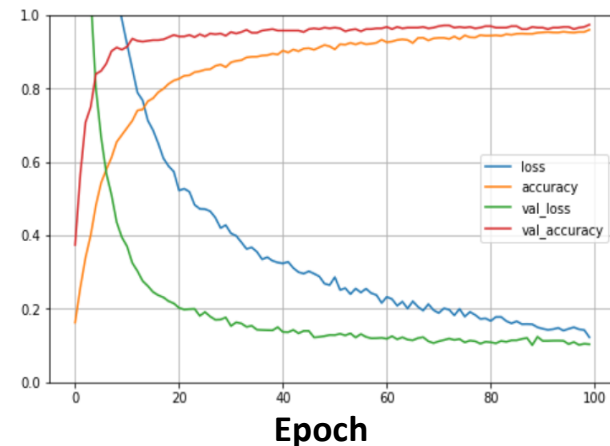
실습 #1) Keras를 활용하여 Deep Neural Network 구현

- 실습 예시에서 사용한 Dense 층을 활용하여 DNN 구현
- Chap.1에서 사용한 "NN_data.csv" 데이터 사용
- label 데이터는 One-Hot Encoding 하여 사용
- Train set, Test set을 분할하여 사용

예상 결과



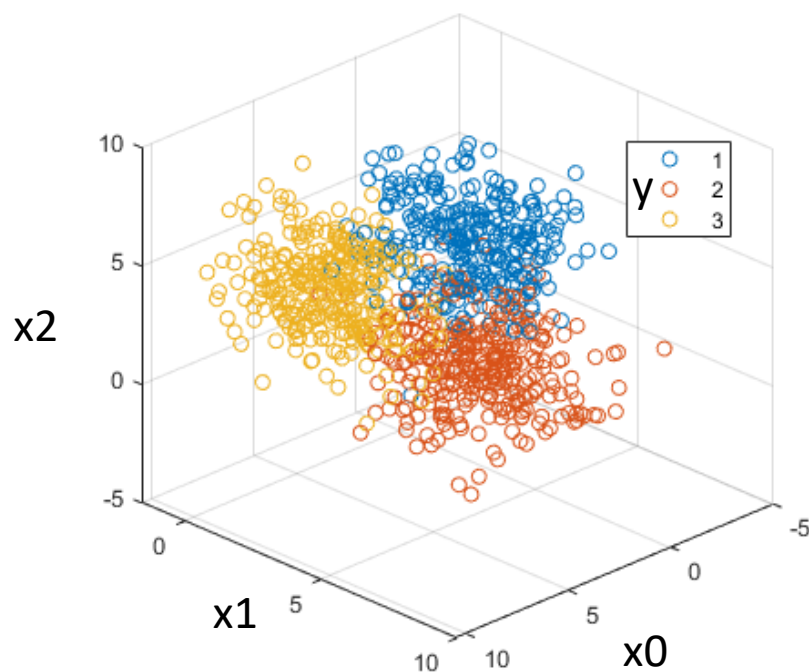
Accuracy
or
Loss



실습 #2) Optimizer 비교 실습

- 실습 #1)에서 여러 Optimizer를 적용
- 각 Optimizer 마다 학습이 잘 되도록 최적의 하이퍼 파라미터 조정(활성화 함수, lr 등등)
- 각 Optimizer 별 학습 결과 비교 분석
- "NN_data"에 가장 최적인 Optimizer가 무엇인지 판별 및 이유 분석

Chap.2 실습 데이터



- 1) Class1 : $(x_0, x_1, x_2) = (2, 4, 6)$ 을 중심으로 노이즈가 추가된 데이터
- 2) Class2 : $(x_0, x_1, x_2) = (4, 6, 2)$ 을 중심으로 노이즈가 추가된 데이터
- 3) Class3 : $(x_0, x_1, x_2) = (6, 2, 4)$ 을 중심으로 노이즈가 추가된 데이터

각 Class별 300개씩 총 900개